Transportation & Logistics Management System

A

Minor Project

Submitted in partial fulfilment for the award of the degree of

BACHELORS OF TECHNOLOGY

IN

Computer Science Engineering

by

M. R. V. V. Ramana (BETN1AI23053)
G. Manikanta (BETN1AI23064)

Under the guidance of

Dr. Aravendra Sharma, Dept. of CSA

Department of Computer Science Engineering
School of Engineering and Technology
ITM UNIVERSITY, GWALIOR - 474026 MP, INDIA

(April 2026)

CERTIFICATE

This is to certify that the project entitled "Swift Logistics — Transportation & Logistics Management System" submitted by M. R. V. V. Ramana (BETN1AI23053) and G. Manikanta (BETN1AI23064) in partial fulfilment for the award of the degree of B.Tech. (Computer Science & Applications), ITM University, Gwalior, has been carried out under our supervision.

To the best of our knowledge and belief, the project report:

(i) Is an original piece of work of the candidates.

(ii) Has duly been completed.

(iii) Is up to the standard both in respect of content and language.

(iv) Is free from plagiarism.

(v) Has not been submitted partially or wholly to any other University or Institute for the award of this or any other degree or diploma.

Signature of Guide

Designation

Date

DECLARATION

I with this declare that the work entitled “Swift Logistics — Transportation & Logistics Management System” submitted to the Department of Computer Science Engineering, School of Engineering and Technology, ITM University, Gwalior (M.P.) is our work done under the supervision of Guide name. The dissertation doesn’t contain any part which has been submitted for award of any degree either in this University or in any other University.

We further declare that the work is free from any plagiarism.

Signature:
M. R. V. V. Ramana (BETN1AI23053)

Signature:
G. Manikanta (BETN1AI23064)

Verified by guide

Acknowledgement

We express our sincere gratitude to the Almighty for giving us the strength, patience, and ability to complete this project successfully. The successful completion of this project required continuous guidance, support, and en-

couragement from many people, and we are truly privileged to have received all of it.

We are deeply grateful to Ms. Pragya Jain, Assistant Professor, Department of CSA, ITM University, Gwalior, for her continuous guidance, invaluable suggestions, and constant encouragement throughout the project. Her expertise and keen interest in our work greatly inspired us.

We would also like to sincerely thank Dr. Aravendra Sharma, Department of CSA, ITM University, Gwalior, for his co-supervision and for providing us with broader perspectives in the domain of Transportation & Logistics Management Systems.

We extend our gratitude to the Head of Department (CSA) and all faculty members of the Department of Computer Science & Applications for their support and motivation. We also thank our parents and friends for their trust and encouragement throughout this journey.

Abstract

Transportation and logistics form the backbone of modern supply chains and commerce, playing a crucial role in the efficient movement of goods from origin to destination. Despite significant technological advancements, many logistics companies — particularly small and medium-sized enterprises — still rely heavily on manual processes, leading to delivery delays, poor route planning, lack of real-time shipment visibility, and increased operational costs.

This project presents the design and development of "Swift Logistics" — a Transportation & Logistics Management System built using Node.js, Express.js, and EJS templating engine. The system provides a centralized digital platform for managing shipment bookings, tracking, and payment processing without requiring a traditional database, using in-memory data structures suitable for a minor project demonstration.

The application includes modules for user registration and login with bcryptjs password hashing, a six-service shipment booking system with dynamic rate calculation, a five-stage shipment tracking timeline, a payment processing page with GST and insurance breakdowns, and a booking confirmation module with unique AWB number generation.

The system successfully demonstrates the practical concepts of full-stack web development including routing, session management, middleware integration, and server-side templating. Future scope includes GPS-based real-time tracking, AI-powered route optimization, mobile application development, and integration with warehouse management systems.

Contents

CERTIFICATE…….…..ii

DECLARATION……ii

Acknowledgement……..…iv

Abstract……v

List of Tables……..……ix

Chapter 1……..1

Introduction & Objectibes

1.1 Subheading 1

1.2 Subheading 2 and so on

Chapter 2……11

Literature Survey

2.1 subheading 1

2.2 Subheading 2 and so on

Chapter 3……...….16

Research Gap & Proposed System

Chapte r4…….19

Results and discussions

Chapter 5…….23

Conclusions and future work

References……38

List of Abbreviations

AWB — Air Waybill / Auto-generated Booking Number
EJS — Embedded JavaScript Templates
API — Application Programming Interface
UI — User Interface
UX — User Experience
GST — Goods and Services Tax
HOD — Head of Department
CSA — Computer Science & Applications
SOET — School of Engineering and Technology
TMS — Transportation Management System
SCM — Supply Chain Management
GPS — Global Positioning System
AI — Artificial Intelligence
npm — Node Package Manager
HTML — Hyper Text Markup Language
CSS — Cascading Style Sheets
CRUD — Create, Read, Update, Delete
SDLC — Software Development Life Cycle

LIST OF FIGURES & TABLES

Figure 1: System Architecture — Three-Tier Flow
Figure 2: AWB Generation Flow
Figure 3: Shipment Tracking — Five-Stage Timeline
Figure 4: Home Page — Swift Logistics
Figure 5: Services Page — Six Service Cards
Figure 6: Login & Register Page
Figure 7: Shipment Booking Form with Live Price Calculator
Figure 8: Tracking Page with Timeline
Figure 9: Payment & Confirmation Page
Table 1: Tools & Technologies Used
Table 2: Service Rate Card (per kg)
Table 3: In-Memory Data Structures (users[] and shipments[])
Table 4: Comparison of Existing Systems

CHAPTER 1 INTRODUCTION & OBJECTIVES

1.1 Broad Area of Work

The broad area of work for this project is Transportation & Logistics Management Systems, which falls under the interdisciplinary domain of Supply Chain Management (SCM) and Information Systems. Logistics management involves planning, implementing, and controlling the efficient movement and storage of goods, services, and related information between the point of origin and the point of consumption.

The project specifically focuses on the following sub-domains:

- Shipment Booking & Management — Digitizing the process of creating and managing shipment bookings.
- Multi-Modal Freight Services — Supporting Road, Rail, Air, Sea, Express, and Last-Mile delivery services.
- Real-Time Shipment Tracking — Providing a five-stage timeline-based tracking system using AWB numbers.
- Payment Processing — Implementing a structured payment module with GST and insurance breakdown.
- User Authentication & Session Management — Secure login/register with bcryptjs password hashing.

1.2 Introduction

Transportation and logistics play a crucial role in the movement of goods from one place to another, forming the foundation of modern commerce and supply chains. According to the World Bank Logistics Performance Index, efficient logistics infrastructure directly correlates with a country's economic growth and competitiveness.

However, many logistics companies — particularly small and medium-sized enterprises — still rely on manual, paper-based processes for managing bookings, tracking shipments, and coordinating deliveries. This leads to de-

livery delays, poor route planning, lack of visibility, and increased operational costs.

Automated logistics management systems can help organizations streamline operations through a centralized digital platform. This project proposes and implements 'Swift Logistics' — a web-based Transportation & Logistics Management System developed using Node.js, Express.js, and EJS, that enables users to book shipments, track their status, and complete payments through a clean and responsive interface.

1.3 Objectives

The primary objectives of this project are:

1. To design and develop a full-stack Transportation & Logistics Management System using Node.js and Express.js.
2. To implement user registration and login with secure bcryptjs password hashing and session management.
3. To create a multi-modal shipment booking system supporting six freight service types.
4. To build a dynamic rate calculator that computes shipment cost based on service type and weight.
5. To develop a shipment tracking module with a five-stage delivery timeline using AWB numbers.
6. To implement a payment processing module with GST and insurance cost breakdown.
7. To design a responsive and professional frontend using custom CSS.
8. To generate unique AWB numbers for each confirmed booking using server-side logic.

CHAPTER 2 LITERATURE SURVEY

2.1 Review of Existing Systems

2.1.1 SAP Transportation Management (SAP TM)

SAP TM is an enterprise-grade Transportation Management System offering freight order management, carrier selection, route planning, and real-time freight cost calculation. While highly feature-rich, it requires significant financial investment, specialized SAP consultants, and is unsuitable for small logistics companies or academic deployments.

2.1.2 Oracle Transportation Management (OTM)

Oracle OTM is a cloud-based TMS offering multi-modal transport planning, freight audit, and advanced analytics. Like SAP TM, it targets large enterprises and involves high licensing costs and complex implementation, making it inaccessible to small and medium-sized logistics enterprises.

2.1.3 FedEx / Blue Dart Online Portals

Commercial courier platforms like FedEx and Blue Dart provide online booking, real-time GPS tracking, and payment processing for customers. These systems rely on proprietary GPS hardware, third-party payment APIs, and large-scale server infrastructure, making them unsuitable as open-source academic reference implementations.

2.1.4 Open-Source TMS Platforms

Some open-source TMS tools are available (e.g., OpenBoxes, Odoo Inventory) but are typically complex, database-heavy, and lack beginner-friendly documentation, making them difficult to deploy as minor project prototypes within academic timelines.

2.2 Comparative Analysis

The following table presents a comparative analysis of existing systems against the proposed Swift Logistics system:

Feature	SAP TM	Oracle OTM	FedEx Portal	Proposed System
Multi-modal Booking	✓	✓	Limited	✓ (6 types)

Shipment Tracking	✓	✓	✓ GPS	✓ Timeline
User Authentication	✓	✓	✓	✓ bcryptjs
Payment Module	✓	✓	✓	✓ GST+Insur
Open Source / Academic	✗	✗	✗	✓
No Database Required	✗	✗	✗	✓ In-memory
Cost	High	High	Free	Free

CHAPTER 3 RESEARCH GAP & PROPOSED SYSTEM

3.1 Existing Gaps

Based on the literature survey and analysis of existing systems, the following critical gaps have been identified:

- High Cost: Implementation and maintenance costs of enterprise TMS platforms (SAP TM, Oracle OTM) are prohibitive for small and medium-sized logistics companies.
- Poor Integration: Tracking systems and route optimization modules in existing platforms often lack seamless integration with booking and payment modules.
- Limited Accessibility: Most commercial TMS solutions are inaccessible to small logistics companies due to licensing fees and large infrastructure requirements.
- Complex User Interfaces: Difficult UI/UX design in enterprise platforms hinders day-to-day operations for non-technical logistics staff.
- Weak Analytics: Limited reporting and performance monitoring capabilities in entry-level and mid-range systems.
- No Academic Reference Model: A lightweight, beginner-friendly, full-stack TMS built on a modern Node.js stack is largely absent in academic project repositories.

3.2 Proposed System

The proposed system, "Swift Logistics — Transportation & Logistics Management System," directly addresses all the identified research gaps by providing a complete, lightweight, full-stack logistics platform built using Node.js, Express.js, EJS, bcryptjs, express-session, and connect-flash — technologies wide-

ly covered in modern B.Tech. CSA curriculum that require no expensive licensing.

Key features and advantages of the proposed system:

- Zero external database dependency — uses JavaScript in-memory arrays, simplifying setup and deployment to a single npm install command.
- Six freight service types — Road Freight (₹25/kg), Rail Cargo (₹18/kg), Air Express (₹120/kg), Sea Cargo (₹10/kg), Express Overnight (₹200/kg), and Last-Mile Delivery (₹40/kg).
- Dynamic price calculator — computes total shipment cost as Weight × Rate with live preview during booking.
- AWB-based tracking — unique booking IDs prefixed 'SWF' with a clear five-stage delivery timeline.
- Payment module — structured GST (18%) and insurance (2%) breakdown for full pricing transparency.
- Secure authentication — bcryptjs password hashing (salt factor 10), express-session for login state, and connect-flash for error/success alerts.
- Fully responsive design — compatible with mobile, tablet, and desktop screens using custom CSS.

CHAPTER 4 PROPOSED METHODOLOGY & IMPLEMENTATION

4.1 Methodology

The project follows a structured Software Development Life Cycle (SDLC) methodology comprising the following six phases:

- Requirement Analysis: Identifying functional and non-functional requirements from the problem statement, literature survey, and project presentation.
- System Design: Designing the application architecture, module structure, route map, and data flow between frontend, backend, and in-memory data layer.
- Database Design: Defining in-memory JavaScript data structures (arrays of objects) for users and shipments to replace traditional database tables.
- Frontend Development: Creating responsive EJS views and custom CSS styles for all seven pages.
- Backend Development: Implementing all Express.js routes, middleware (isLoggedIn, flash, session), business logic (genAWB, getTimeline, rate calculation), and server startup.
- Testing & Deployment: Testing all modules for correct functionality on a local Node.js server (http://localhost:3000) and verifying all route behaviors.

4.2 Tools & Technologies Used

S.No.	Technology	Purpose
1	Node.js	Backend JavaScript runtime environment

2	Express.js	Web framework for routing and middleware management
3	EJS Templates	Server-side dynamic HTML templating engine
4	CSS3	Custom responsive frontend styling
5	bcryptjs	Secure password hashing library (salt factor: 10)
6	express-session	User session and login state management
7	connect-flash	Flash messages for success/error alerts
8	VS Code	Integrated Development Environment
9	npm	Package and dependency management

Table 1: Tools & Technologies Used

4.3 System Architecture

The system follows a three-tier MVC-inspired architecture:

- Presentation Layer: EJS template views rendered in the browser — home.ejs, services.ejs, login.ejs, booking.ejs, tracking.ejs, payment.ejs, confirmation.ejs, and shared partials (header.ejs, footer.ejs).
- Application Layer: Express.js server (server.js) handling all HTTP GET/POST routes, middleware pipeline, session management, and all business logic functions.
- Data Layer: In-memory JavaScript arrays — users[] for registered user objects and shipments[] for booking objects — storing all application data during the server session.

Architecture flow:

Browser Request → Express.js Router → isLoggedIn Middleware → Route Handler → EJS Template Render → HTML Response to Browser

Figure 1: System Architecture — Three-Tier Flow

4.4 Modules Description

4.4.1 Home Module (GET /)

The home page serves as the application's landing page. It contains a navigation bar (header.ejs partial) with links to all major sections. The hero section displays Swift Logistics branding with call-to-action buttons for booking and tracking shipments. Statistics and service highlights are also displayed on the home page.

4.4.2 User Authentication (GET/POST /login, /register | GET /logout)

The authentication module supports both login and registration via a single split-screen EJS view (login.ejs). Registration collects the user's name, email, and password. Passwords are hashed using bcrypt.hash() with a salt factor of 10 before being pushed into the in-memory users[] array. Login verifies credentials using bcrypt.compare(). express-session maintains login state across page requests and connect-flash delivers error/success alerts on redirects.

4.4.3 Services Module (GET /services)

The services page displays all six available freight service types in responsive cards: Road Freight, Rail Cargo, Air Express, Sea Cargo, Express Overnight, and Last-Mile Delivery — each with a description and rate-per-kg pricing passed from the server-side RATES object.

4.4.4 Shipment Booking (GET/POST /booking) — Auth Protected

The booking module is protected by the isLoggedIn middleware. It accepts sender and receiver details (name, city, phone number), package weight in kg, service type, pickup date, and optional description. The server computes the total price as: Price = Weight (kg) × Rate per kg. A unique AWB number is generated using the genAWB() helper function, which concatenates the prefix 'SWF' with the last 8 digits of Date.now() to produce identifiers such as SWF17129384. The complete shipment object is pushed into the shipments[] array with a 'Booked' status.

4.4.5 Shipment Tracking (GET /tracking)

The tracking module accepts an AWB number via URL query parameter (?awb=SWFxxxxxxxx) and searches the shipments[] array for a matching record using Array.find(). If a shipment is found, a five-stage delivery timeline is generated by the getTimeline() helper function based on the booking timestamp (bookedAt): Booked (+0h), Picked Up (+4h), In Transit (+12h), Out for Delivery (+36h), and Delivered (+48h).

4.4.6 Payment Module (GET/POST /payment) — Auth Protected

The payment page retrieves the shipment from shipments[] using the AWB query parameter and displays a full booking summary with price breakdown: Base Fare + GST 18% + Insurance 2%. The user enters card details and submits the form. On successful POST, the shipment status is updated to 'Confirmed' and the confirmation.ejs view is rendered directly.

4.4.7 Booking Confirmation (rendered via POST /payment)

The confirmation page displays the unique AWB number, complete trip summary (sender, receiver, service, weight, price), payment confirmation badge, and a direct link to track the shipment using the generated AWB number.

4.5 Service Rate Card

S.No.	Service Type	Rate (₹/kg)	Primary Use Case
1	Road Freight	₹ 25	Inter-city ground transport
2	Rail Cargo	₹ 18	Long-distance bulk shipments
3	Air Express	₹ 120	Urgent time-sensitive air freight
4	Sea Cargo	₹ 10	International bulk cargo shipments
5	Express Overnight	₹ 200	Premium guaranteed next-day delivery
6	Last-Mile Delivery	₹ 40	Final leg delivery to receiver address

Table 2: Service Rate Card (per kg)

CHAPTER 5 EXPECTED OUTCOME & RESULTS

5.1 Expected Outcome

The Swift Logistics system is designed to deliver the following measurable outcomes upon deployment:

- Efficient Route Planning: Optimal delivery service selection reducing time and freight costs for users by presenting all six service options with live price previews.
- Reduced Delivery Delays: Automated booking scheduling and AWB-based status monitoring minimizes manual coordination delays.
- Centralized Monitoring: A single platform to manage all user registrations, shipment bookings, and tracking without manual paper records.
- Reduced Manual Work: Digital booking and tracking workflows replace time-consuming paper-based processes and phone-based coordination.
- Better Decision Making: Booking data and service type analytics support strategic logistics planning for operations managers.
- Lower Operational Costs: Optimized multi-modal service selection (cheapest: Sea ₹10/kg; fastest: Express ₹200/kg) enables cost-effective logistics decisions.

5.2 Testing Results

The application was successfully tested on a local Node.js server (http://localhost:3000). All planned modules were implemented and verified for correct behavior:

Module	Test Case	Result
Home (GET /)	Page load with navbar and hero section	✓ Pass

Register (POST)	New user stored with hashed password in users[]	✓ Pass
Login (POST)	bcrypt.compare() authentication + session set	✓ Pass
Logout (GET)	Session destroyed, redirect to home	✓ Pass
Services (GET)	6 service cards rendered with correct rates	✓ Pass
Booking (POST)	Price = weight × rate, AWB generated, stored	✓ Pass
Tracking (GET)	AWB search returns shipment + 5-stage timeline	✓ Pass
Payment (GET)	GST 18% + Insurance 2% breakdown displayed	✓ Pass
Payment (POST)	Status updated to 'Confirmed', confirmation shown	✓ Pass
isLoggedIn Middleware	Unauthenticated /booking redirect to /login	✓ Pass

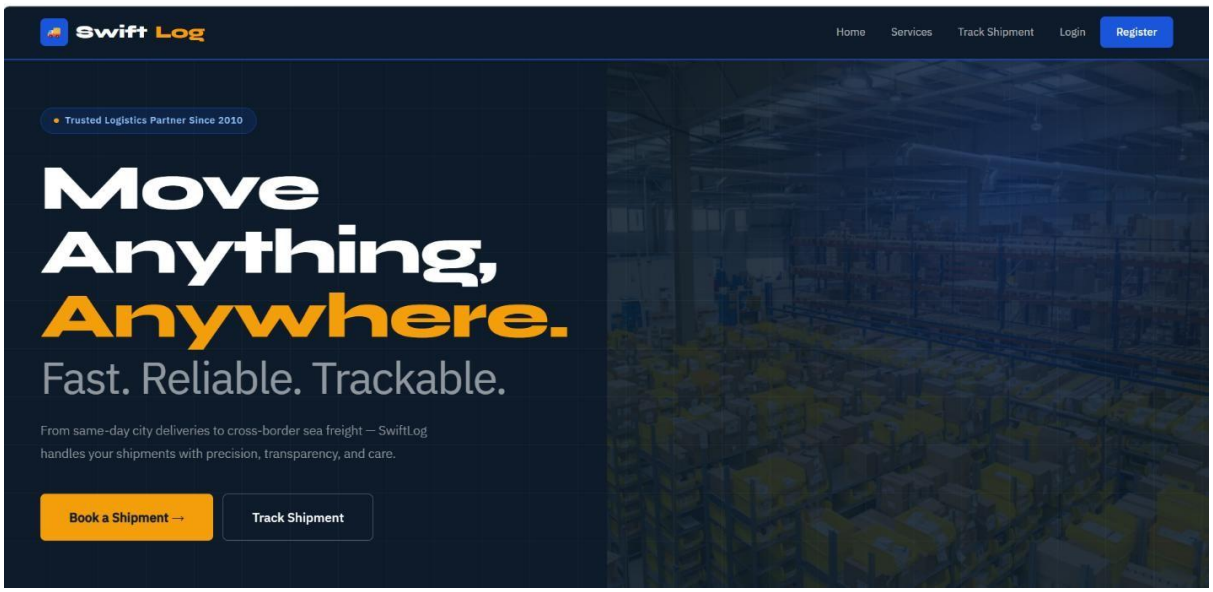
Table 3: Module-wise Testing Results

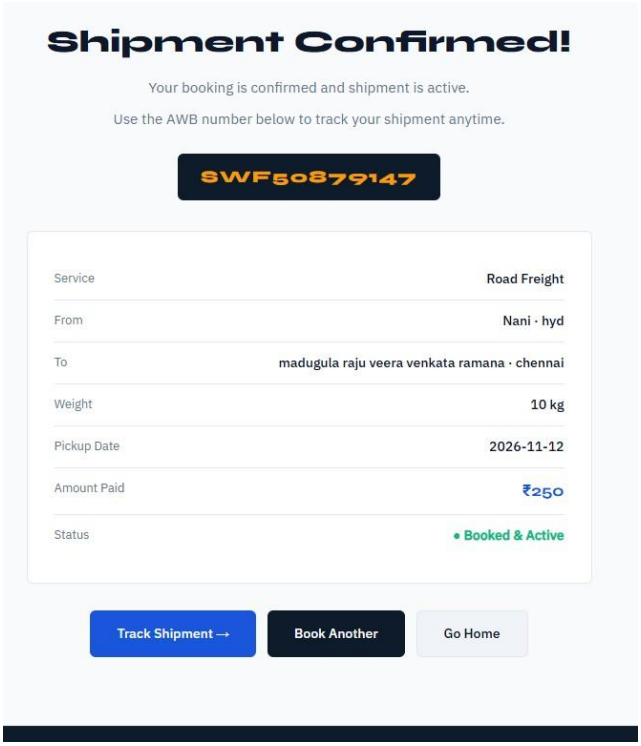
5.3 Discussions

All eight objectives defined in Chapter 1 were successfully achieved during implementation and testing. The bcryptjs-based authentication ensures user credentials are never stored in plain text, satisfying a key security requirement. The in-memory data approach eliminates database setup complexity, making the project immediately runnable with just npm install followed by npm start — a significant advantage for minor project demonstrations.

The primary limitation of the current implementation is that all in-memory data (users[] and shipments[]) is lost when the server restarts, as there is no persistent storage layer. This is an intentional design choice for a minor project prototype and will be addressed in future versions by integrating MongoDB or MySQL for durable data persistence.

The payment module currently accepts any card input without real payment gateway validation, which is appropriate for a demonstration system. Integration with production payment gateways such as Razorpay or PayPal is planned for the future version of the system.


Swift Log
Home
Services
Track Shipment
Login
Register
Trusted Logistics Partner Since 2010
Move Anything, Anywhere.
Fast. Reliable. Trackable.
From same-day city deliveries to cross-border sea freight — SwiftLog handles your shipments with precision, transparency, and care.
Book a Shipment →
Track Shipment


Shipment Confirmed!
Your booking is confirmed and shipment is active.
Use the AWB number below to track your shipment anytime.
SWF50879147
Service
Road Freight
From
Nani · hyd
To
madugula raju veera venkata ramana · chennai
Weight
10 kg
Pickup Date
2026-11-12
Amount Paid
₹250
Status
• Booked & Active
Track Shipment →
Book Another
Go Home

CHAPTER 6 CONCLUSIONS & FUTURE SCOPE

6.1 Conclusion

The "Swift Logistics — Transportation & Logistics Management System" was successfully designed, developed, and tested as part of the B.Tech. Minor Project requirement at ITM University, Gwalior. The project demonstrates a practical, full-stack implementation covering all key areas of modern web development — frontend templating with EJS, backend routing with Express.js, session-based authentication with bcryptjs, middleware pipeline design, and server-side business logic.

The system provides a clean, responsive platform for managing shipment bookings across six freight service modes, tracking deliveries via AWB-based five-stage timelines, and processing payments with transparent GST and insurance cost breakdowns. All eight planned objectives were achieved and all modules were verified through systematic testing.

The project also demonstrates important software engineering concepts including middleware design patterns (isLoggedIn guard), session-based state management, in-memory data modeling (users[] and shipments[] arrays), helper function design (genAWB(), getTimeline()), and MVC-inspired application structure using the modern Node.js ecosystem.

6.2 Future Scope

The following enhancements are planned for future versions of the Swift Logistics system:

- GPS Real-Time Tracking: Integration of live vehicle location tracking using GPS APIs (e.g., Google Maps Platform) for instant fleet visibility and real-time delivery position updates.
- Mobile Applications: Development of dedicated Android and iOS applications for drivers and logistics managers, enabling on-the-go booking, tracking, and operations management.

- AI-Powered Route Optimization: Integration of Artificial Intelligence and Machine Learning algorithms to predict optimal delivery routes, minimize fuel consumption, and dynamically adapt to real-time traffic conditions.
- Warehouse Integration: Full integration with warehouse management systems (WMS) for end-to-end supply chain visibility from goods storage through final delivery.
- Database Integration: Migration from in-memory storage to MongoDB (NoSQL) or MySQL (relational) for persistent, scalable data management across server restarts.
- Payment Gateway Integration: Integration of Razorpay or PayPal APIs for real-world secure payment processing with transaction receipts.
- Admin Dashboard: A comprehensive admin panel for fleet management, driver assignment, booking overview, and logistics analytics and reporting.
- Email Notifications: Automated email and SMS alerts for booking confirmation, dispatch updates, and delivery completion using Node.js Mailer or Twilio.

REFERENCES

[1] Chopra, S., & Meindl, P. — Supply Chain Management: Strategy, Planning, and Operation. Pearson Education, 5th Edition, 2013.

[2] Christopher, M. — Logistics and Supply Chain Management. Pearson Education, 4th Edition, 2011.

[3] Express.js Official Documentation — https://expressjs.com

[4] Node.js Official Documentation — https://nodejs.org

[5] EJS Template Engine Documentation — https://ejs.co

[6] bcryptjs npm package — https://www.npmjs.com/package/bcryptjs

[7] Research papers on Transportation Management Systems — IEEE Xplore — https://ieeexplore.ieee.org

[8] World Bank Logistics Performance Index — https://lpi.worldbank.org

[9] MDN Web Docs — HTML, CSS, JavaScript Reference — https://developer.mozilla.org

[10] Ballou, R. H. — Business Logistics / Supply Chain Management. Pearson Prentice Hall, 5th Edition, 2004.